

CO2 ASSESSMENT TOOL 3

NAME : **UDHAYA KUMAR U**

REG NO : **192424332**

COURSE CODE : **CSA 0606 DESIGN AND ANALYSIS ALGORITHMS**

1.)Debugging Selection Sort Code (Minimum Candidate Compared Against Wrong Position)

The following Selection Sort implementation compares each new element against `arr[i]`

instead of the current minimum candidate, which causes the algorithm to ignore smaller

values found earlier in the same pass. Carefully analyze why the minimum element is not

tracked correctly across the inner loop. Apply systematic debugging so that every comparison

is made against the current minimum index. Optimize the implementation by skipping swaps

when the selected minimum is already in its correct position. Analyze the time complexity of

the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        min_idx = i
```

```
        for j in range(i+1, n):
```

```
            if arr[j] < arr[i]: # ERROR
```

```
                min_idx = j
```

```
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

```
    return arr
```

A.) 1. Objective

The objective is to debug the given Selection Sort implementation by correcting the comparison logic so that the algorithm always tracks the current minimum element correctly. The implementation is also optimized by avoiding unnecessary swaps when the minimum element is already in its correct position.

2. Problem Identification

The given code incorrectly compares each element with `arr[i]` instead of the current minimum element `arr[min_idx]`.

Incorrect Code

```
if arr[j] < arr[i]: # ERROR
    min_idx = j
```

Impact

- The smallest element found during the search may not be retained.
- The algorithm may select the wrong minimum element.
- The array may not be sorted correctly.
- Unnecessary swaps are performed even when the element is already in the correct position.

3. Root Cause Analysis

Selection Sort works by continuously updating the position of the current minimum element during each pass.

The error occurs because the comparison is made with the first element of the pass (`arr[i]`) instead of the current minimum element (`arr[min_idx]`).

Incorrect

```
if arr[j] < arr[i]
```

Correct

```
if arr[j] < arr[min_idx]
```

As a result, the algorithm fails to remember smaller elements that were found earlier in the same pass.

4. Debugging Methodology

The following debugging steps were followed:

- Reviewed the Selection Sort algorithm and compared it with the given implementation.
- Traced the values of `i`, `j`, and `min_idx` during each iteration.
- Identified that comparisons were being made with the wrong array element.
- Replaced `arr[i]` with `arr[min_idx]`.
- Added a condition to skip swapping when `min_idx == i`.
- Tested the corrected program using different input arrays (random, sorted, reverse-sorted, and duplicate values).

- **Verified that the output was correctly sorted for all test cases.**

5. Solution Implemented

Changes Made

- 1. Updated the comparison:**

```
if arr[j] < arr[min_idx]:
```

- 2. Added swap optimization:**

```
if min_idx != i:
```

```
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

- 3. Included comments to improve code readability and maintainability.**

Corrected Code

```
def selection_sort(arr):
```

```
    n = len(arr)
```

```
    # Traverse through the array
```

```
    for i in range(n - 1):
```

```
        # Assume current element is the minimum
```

```
        min_idx = i
```

```
        # Find the smallest element
```

```
        for j in range(i + 1, n):
```

```
            # Compare with current minimum
```

```
            if arr[j] < arr[min_idx]:
```

```

        min_idx = j

    # Swap only if needed
    if min_idx != i:
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr

arr = [64, 25, 12, 22, 11]

print("Original Array :", arr)

sorted_arr = selection_sort(arr)

print("Sorted Array  :", sorted_arr)

```

6. Optimization Techniques

The following optimization was applied:

- **Correct Minimum Tracking:** Compared each element with the current minimum (**arr[min_idx]**) instead of the first element of the pass.
- **Skip Unnecessary Swaps:** Performed a swap only when the minimum element was not already in its correct position.
- **Improved Readability:** Added meaningful comments and maintained a clean algorithm structure.

Although the optimization does not reduce the asymptotic time complexity, it decreases the number of unnecessary swap operations and improves practical performance.

7. Performance Evaluation

Metric	Before Debugging	After Debugging
Sorting Correctness	Incorrect for some inputs	Correct for all tested inputs
Comparisons	$O(n^2)$	$O(n^2)$
Swap Operations	Swapped every pass	Swapped only when required
Time Complexity	$O(n^2)$	$O(n^2)$
Space Complexity	$O(1)$	$O(1)$
Code Reliability	Low	High

Example

Input

[64, 25, 12, 22, 11]

Before Debugging

May produce incorrect ordering for some datasets.

After Debugging

[11, 12, 22, 25, 64]

The corrected algorithm consistently produces the correct sorted output while avoiding unnecessary swaps.

8. Conclusion

The Selection Sort implementation was successfully debugged by correcting the comparison logic to track the current minimum element throughout each pass. An additional optimization was introduced to skip unnecessary swaps when the selected minimum was already in its correct position. The corrected algorithm reliably sorts the array, improves execution efficiency by reducing redundant swap operations, and maintains $O(n^2)$ time complexity with $O(1)$ space complexity. Overall, the debugging and optimization enhance the correctness, reliability, and practical performance of the sorting algorithm.